Range vs Hash Partitioning

scans, hot spots and compound keys

Watch the 3-min video

In one line: range partitioning keeps keys sorted in contiguous chunks, so range scans are cheap but sequential keys create write hot spots; hash partitioning spreads keys evenly but makes range queries fan out; compound keys hash one part and sort by the other to get some of both.

1 Two systems, two different answers

You run two systems. A **logging platform**, where engineers ask for the last hour of logs. And a **user service**, where every request looks up one user by ID. Both are too big for one machine, so both must be split into **partitions** (shards) spread across nodes.

Should both be partitioned the same way? No. The right scheme depends on the queries you run. The two basic choices are **range** and **hash** partitioning.

2 Range partitioning: sorted chunks

Range partitioning gives each partition a **contiguous range of the key**, like the pages of a dictionary: A to F on one node, G to M on the next. Or one partition per day. Keys stay **sorted**, so a range scan (everything from 09:00 to 10:00) reads just one or a few partitions.

Bigtable, HBase, Spanner and MongoDB's ranged sharding work this way.

The catch is hot spots. If the key increases over time, like a timestamp or an auto-increment ID, **every new write lands in the last partition**. One node takes all the writes while the others sit idle. Uneven key ranges (a very popular first letter) cause the same problem.

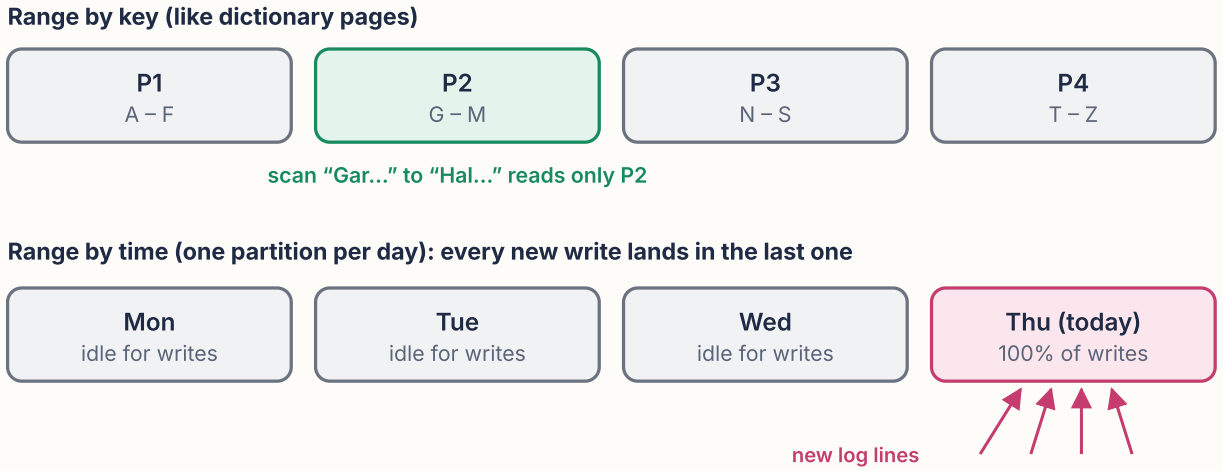


Figure 1. Top: a scan within one key range touches one partition. Bottom: with a time key, today's partition takes every write while older ones only serve reads.

3 Hash partitioning: even spread

Hash partitioning runs the key through a **hash function** first, and the hash decides the partition. Even sequential keys (1001, 1002, 1003) get scattered, so data and load spread evenly. No last-partition hot spot.

The price: neighbours in key order end up on different partitions, so a **range query must ask all of them** (scatter-gather) and merge the results. Cassandra, DynamoDB and MongoDB's hashed sharding use this.

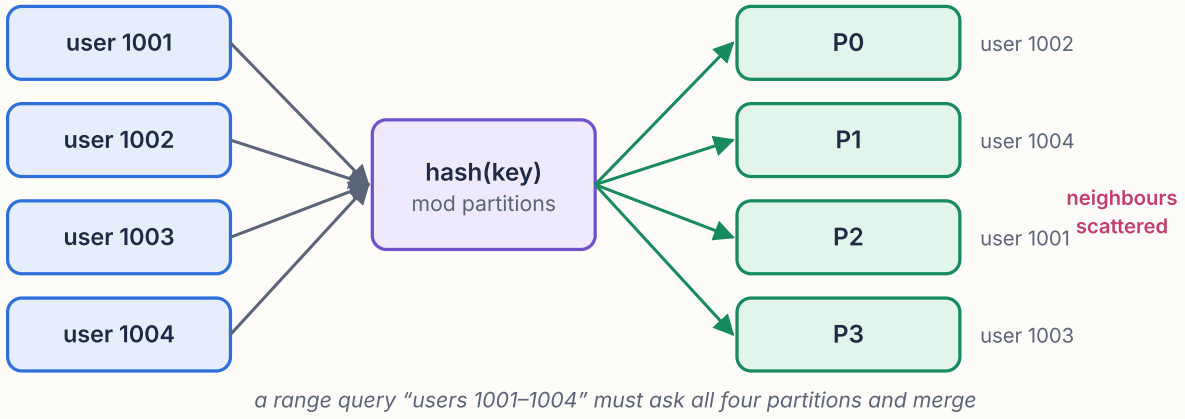


Figure 2. The hash scatters consecutive keys. Point lookups go to exactly one partition; range queries go to every partition.

4 Compound keys: some of both

You can combine the two. **Hash the first part** of the key to choose the partition, and **sort by the second part** inside it. In Cassandra, the first part is the *partition key* and the rest are *clustering columns*.



Figure 3. Hashing sensor_id spreads sensors over nodes; within each partition (highlighted: sensor 42) rows are ordered by timestamp, so one sensor's time range is one sorted slice.

In CQL:

```
CREATE TABLE readings (
  sensor_id text,
  ts        timestamp,
  value     double,
  PRIMARY KEY ((sensor_id), ts)   -- hash sensor_id, sort by ts
);

SELECT * FROM readings
 WHERE sensor_id = 's-42'
   AND ts >= '2026-11-04 09:00' AND ts < '2026-11-04 10:00';
-- one partition, one sorted slice: fast
```

Sensors spread evenly across nodes, while one sensor's readings stay sorted by time. A range scan works **within** one partition key; a range across many sensors still fans out.

5 Back to the two systems

WORKED EXAMPLE

- **Logs:** range partition by time. The last-hour query hits one partition, and dropping a whole old day is cheap. To stop one partition taking every write, add a small hash prefix inside each day, such as `hash(source) % 8`. Today's writes now spread over 8 buckets; a last-hour query reads 8 small slices instead of one, still far fewer than the whole cluster.
- **Users:** hash partition by user ID for an even spread. Every request is a point lookup by ID. Nobody asks for “users 50 to 60”, so losing range scans costs nothing.

	Range	Hash	Compound (hash + sort)
Point lookup	one partition	one partition	one partition
Range scan on key	one or a few partitions	all partitions	one partition, within one partition key
Spread of writes	hot spots with sequential keys	even	even across partition keys
Examples	HBase, Bigtable, Spanner	DynamoDB, Cassandra	Cassandra, DynamoDB (partition + sort key)

COMMON MISTAKES

- Range partitioning on a timestamp or auto-increment ID with no prefix: one hot partition takes every write.
- Hash partitioning a table whose main query is a time-range scan: every query hits every node.
- Choosing the scheme from the data shape instead of from the queries you actually run.
- A compound partition key so coarse (e.g. country) that one partition grows huge and hot.

6 Key terms

Partition (shard): a subset of rows held by one node

Hot spot: one partition gets most of the load

Clustering / sort key: orders rows inside a partition

Range partitioning: contiguous key ranges per partition

Scatter-gather: ask all partitions, merge the answers

Compound key: partition key + sort key

Hash partitioning: hash of the key picks the partition

Partition key: the part of the key that picks the partition

Hash prefix: small bucket number to spread sequential writes

7 Interview questions

Q1. Compare range and hash partitioning. BASIC

Range keeps keys sorted in contiguous chunks, so range scans touch one or a few partitions, but sequential keys like timestamps send all writes to the last partition. Hash spreads keys evenly and avoids that hot spot, but a range query must fan out to every partition.

Q2. Why does an auto-increment key cause a hot spot under range partitioning? BASIC
Each new ID is larger than the last, so every insert falls in the highest range. With 10 nodes, one node takes 100% of writes and nine take none.

Q3. What is a compound key in Cassandra? BASIC
PRIMARY KEY ((sensor_id), ts): sensor_id is hashed to pick the partition (and node); ts orders rows inside it. One sensor's time range is one sorted slice on one partition.

Q4. Design partitioning for a log store queried by "last hour". SCENARIO
Range by time (per hour or day) so recent queries hit few partitions and old data drops cheaply. Add a prefix such as hash(source) % 8 inside each period so writes spread over 8 buckets; the query reads those 8 buckets.

Q5. A user service only does lookups by user ID. Which scheme? SCENARIO
Hash by user ID. Every request is a point lookup to one partition, load is even, and no one needs ID ranges.

Q6. Your chat app stores messages. Users open a conversation and scroll back in time. SCENARIO
Partition key = conversation_id (hashed, even spread), sort key = message time or a time-ordered message ID. Loading the last 50 messages is one partition, one sorted slice.

Q7. With hash partitioning, is a range query impossible? TRICKY
No, just expensive: it goes to every partition and the results are merged, so latency follows the slowest partition. With 50 partitions, one slow node slows every range query.

Q8. Does a hash prefix fix the hot spot for free? TRICKY
No. Writes spread over N buckets, but every read for a time range now reads N buckets and merges them. Keep N small (e.g. 8) and add it only where the write rate needs it.

Next: SD-E-024 · Rebalancing & Hot Partitions backendroadmap.com